

Lösungsvorschlag zu Übungsblatt 7

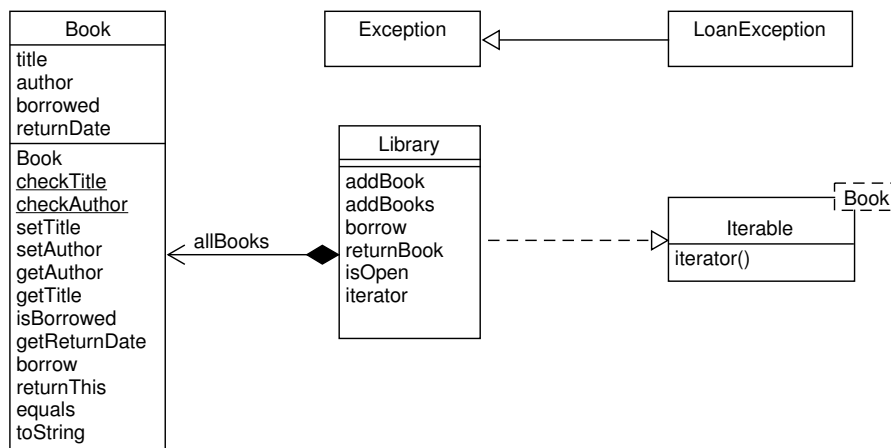
Abgabe: 08.06.2026, 10:00 Uhr (im Digicampus via VIPS: .java-Dateien für Code, .uxf für UML, .pdf für alles andere)

- Dieses Übungsblatt muss im Team abgegeben werden (Einzelabgaben sind nicht erlaubt!).
- Die **Zeitangaben** geben zur Orientierung an, wie viel Zeit für eine Aufgabe später in der Klausur vorgesehen wäre; gehen Sie davon aus, dass Sie zum jetzigen Zeitpunkt wesentlich länger brauchen und die angegebene Zeit erst nach ausreichender Übung erreichen.

* leichte Aufgabe / ** mittelschwere Aufgabe / *** schwere Aufgabe

Aufgabe 25 (*Eigene Klasse implementieren, 25 Minuten*)

In dieser Aufgabe sollen Sie eine weitere Klasse nach einer Systembeschreibung implementieren und diese mit einer eigenen Programmklasse testen.



a) (**, 20 Minuten)

Implementieren Sie basierend auf folgender Systembeschreibung eine eigene Klasse **Library**, mit der das Ausleihen von Büchern verwaltet werden kann. Verwenden Sie für die Bücher die vorgegebene Klassen **Book** und **IllegalLoanException**, die Sie im Digicampus finden.

*Eine Bibliothek verwaltet Bücher, von denen keines doppelt vorkommen darf. Beim Erstellen der Bibliothek kann eine Sammlung an ausleihbaren Büchern übergeben werden, ansonsten hat die Bibliothek zu Beginn keine Bücher. Auch nach dem Erstellen sollen sowohl einzelne Bücher als auch ganze Listen von Büchern der Liste der Bibliothek hinzugefügt werden können. Auf die interne Liste der Bücher soll nicht zugegriffen werden können. Möchte man ein Buch von der Bibliothek ausleihen, so übergibt man den Titel des gewünschten Buchs als String, die Bibliothek sucht in ihrer Liste von Büchern nach einem entsprechende Buch, markiert es als ausgeliehen und gibt das Rückgabedatum aus. Tritt dabei ein Fehler auf, wird dieser an den Aufrufer weitergereicht. Ist ein Buch nicht vorhanden, soll kein Fehler verursacht werden. Um ein Buch zurückzugeben, wird wieder der Titel übergeben, die Bibliothek überprüft, ob ein Buch mit diesem Titel in ihrer Liste enthalten ist und gibt aus, ob das Buch rechtzeitig zurückgegeben wurde oder nicht. Tritt dabei ein Fehler auf, wird dieser an den Aufrufer weitergereicht. Wird versucht, ein Buch zurückzugeben, dass nicht in der Liste enthalten ist, soll eine **IllegalLoanException** mit geeigneter Fehlerbeschreibung verursacht werden. Zusätzlich soll die Bibliothek eine Möglichkeit anbieten, auf allen Büchern nacheinander Operationen ausführen zu können. Außerdem hat die Bibliothek Öffnungszeiten: Sie öffnet um 08:30 Uhr und schließt um 22:00 Uhr. Es soll möglich sein zu bestimmen, ob die Bibliothek jetzt gerade offen hat.*

Lösung:

```
import java.time.LocalDate;
import java.time.LocalTime;
import java.util.ArrayList;
import java.util.Iterator;

public class Library implements Iterable<Book> {

    private final ArrayList<Book> list;

    public Library() {
        list = new ArrayList<>();
    }

    public Library(ArrayList<Book> list) {
        this.list = list;
    }

    public void addBook(Book b) {
        list.add(b);
    }

    public void addBooks(ArrayList<Book> books) {
        list.addAll(books);
    }

    public void borrow(String title) throws IllegalLoanException{
        for (Book b : list) {
            if (b.getTitle().equals(title)) {
                b.borrow();
                System.out.println("Return date: " + b.getReturnDate());
            }
        }
    }
}
```

```

    public void returnBook(String title) throws IllegalLoanException {
        for (Book b : list) {
            if (b.getTitle().equals(title)) {
                if (LocalDate.now().isAfter(b.getReturnDate())) {
                    System.out.println("Return date for this book has already passed!");
                } else {
                    System.out.println("Book has been returned in time.");
                }
                b.returnThis();
                return;
            }
        }
        throw new IllegalLoanException("This book is not available in this library.");
    }

    public Iterator<Book> iterator() {
        return list.iterator();
    }

    public boolean isOpen() {
        return !LocalTime.now().isBefore(LocalTime.of(8, 30)) &&
            ↪ !LocalTime.now().isAfter(LocalTime.of(22, 0));
    }
}

```

b) (*, 5 Minuten)

Schreiben Sie ein übersetzbare Java-Programm, das

- eine Bibliothek `bib` ohne Parameter erzeugt,
- `bib` folgende Bücher als anonyme¹ Objekte hinzufügt:
 - Christian Ullenboom: Java ist auch eine Insel
 - Robert Sedgewick: Einführung in die Programmierung mit Java
 - Marc-Uwe Kling: Qualityland
 - David Eddings: Kind der Prophezeiung
- überprüft und ausgibt, ob die Bibliothek gerade geöffnet ist,
- alle Bücher ausgibt, deren Titel “Java“ enthält,
- versucht, das Buch “Java ist auch eine Insel“ auszuleihen,
- versucht, das Buch “Qualityland“ auszuleihen,
- versucht, das Buch “Qualityland“ zurückzugeben und
- versucht, das Buch “Kind der Prophezeiung“ zurückzugeben.

Treten Fehler auf, so soll deren Fehlermeldung ausgegeben werden.

¹ *Anonym* bedeutet, dass keine Variable das Objekt referenziert.

Lösung:

```
public class LibraryMain {
    public static void main(String[] args) {

        Library bib = new Library();
        bib.addBook(new Book("Christian Ullenboom", "Java ist auch eine Insel"));
        bib.addBook(new Book("Robert Sedgewick", "Einfuehrung in die Programmierung mit
        ↪ Java"));
        bib.addBook(new Book("Marc-Uwe Kling", "Qualityland"));
        bib.addBook(new Book("David Eddings", "Kind der Prophezeiung"));
        System.out.println("Hat die Bibliothek gerade geoeffnet: " + bib.isOpen());
        bib.forEach(b -> {
            if (b.getTitle().contains("Java")) {
                System.out.println(b);
            }
        });
        try {
            bib.borrow("Java ist auch eine Insel");
            bib.borrow("Qualityland");
            bib.returnBook("Qualityland");
            bib.returnBook("Kind der Prophezeiung");
        } catch (IllegalLoanException e) {
            System.out.println("Fehler: " + e.getMessage());
        }
    }
}
```

Aufgabe 26 (Eigene Klasse implementieren, 33 Minuten)

a) (**, 32 Minuten)

Implementieren Sie gemäß folgender Systembeschreibung eine eigene Klasse **BankAccount** zur Verwaltung von Bankkonten.

Bankkonten haben den Namen des Kontohalters, eine Kontonummer, ein Eröffnungsdatum sowie einen Kontowert. Der Name des Kontoinhabers ist ein String und muss aus mindestens einem Wort bestehen, das mit einem lateinischen Großbuchstaben beginnt, gefolgt von beliebig vielen lateinischen Kleinbuchstaben. Der Name des Kontoinhabers kann aus mehreren Worten, getrennt durch Leerzeichen bestehen. Der Name des Kontoinhabers kann jederzeit geändert und gelesen werden. Die Kontonummer ist ein String, die aus genau 12 Großbuchstaben und Ziffern besteht, z.B. "A12B34C56D78". Die Kontonummer wird beim Erstellen des Bankkontos festgelegt und kann danach nicht mehr geändert werden. Wird für den Namen oder die Kontonummer ein ungültiger Wert übergeben, so wird eine geeignete ungeprüfte Ausnahme geworfen. Das Eröffnungsdatum entspricht dem Tag, an dem das Bankkonto erstellt wurde und kann nicht geändert werden. Der Kontostand ist eine reelle Zahl und ist zu Beginn 0. Danach kann der Kontostand um einen übergebenen Betrag (positiv oder negativ) verändert werden. Alle Attribute des Bankkontos sind öffentlich lesbar. Zwei Bankkonten sind genau dann gleich, wenn ihre Kontonummern gleich sind. Wählen Sie eine geeignete Darstellung des Bankkontos als String.

Verwenden Sie für die Attribute die Namen `accountHolderName`, `accountNumber`, `openingDate` und `balance`.

Hinweis: Eine elegante Möglichkeit zur Überprüfung des Namens und der Kontonummer sind die in der Klasse **Pattern** beschriebenen regulären Ausdrücke zur Überprüfung von Strings. Benutzen Sie außerdem die Tatsache, dass Klammern in regulären Ausdrücken Teile des Ausdrucks gruppieren können: `"ab+"` matcht `"ab"` sowie `"abb"` usw., während `"(ab)+"` `"ab"`, `"abab"` usw. matcht. Es ist aber genauso möglich, die Überprüfungen ohne reguläre Ausdrücke zu machen.

Lösung:

```
import java.time.LocalDate;

public class BankAccount {
    private String accountHolderName;
    private String accountNumber;
    private final LocalDate openingDate = LocalDate.now();
    private double balance = 0;

    public BankAccount(String accountHolderName, String accountNumber) {
        this.setAccountHolderName(accountHolderName);
        this.setAccountNumber(accountNumber);
    }

    private static boolean checkAccountHolderName(String accountHolderName) {
        return accountHolderName != null &&
            accountHolderName.matches("[A-Z][a-z]*( [A-Z][a-z]*)*");
        /* if (accountHolderName == null || accountHolderName.length() == 0) {
            return false;
        }
        String[] names = accountHolderName.split(" ");
        for (String name : names) {
            if (name.length() == 0 || name.charAt(0) < 'A' || name.charAt(0) > 'Z') {
                return false;
            }
            for (int i = 1; i < name.length(); i++) {
                if (name.charAt(i) < 'a' || name.charAt(i) > 'z') {
                    return false;
                }
            }
        }
        return true; */
    }

    private static boolean checkAccountNumber(String accountNumber) {
        return accountNumber != null && accountNumber.matches("[A-Z\\d]{12}");
        /* if (accountNumber == null || accountNumber.length() != 12) {
            return false;
        }
        for (int i = 0; i < accountNumber.length(); i++) {
            char c = accountNumber.charAt(i);
            if (!Character.isDigit(c) || c < 'A' || c > 'Z') {
                return false;
            }
        }
        return true; */
    }

    public void setAccountHolderName(String accountHolderName) {
        if (checkAccountHolderName(accountHolderName)) {
            this.accountHolderName = accountHolderName;
        } else {
            throw new IllegalArgumentException("Wrong name format");
        }
    }

    public void changeBalance(double value) {
        this.balance += value;
    }

    private void setAccountNumber(String accountNumber) {
        if (!checkAccountNumber(accountNumber)) {
            throw new IllegalArgumentException("wrong account number format");
        }
        this.accountNumber = accountNumber;
    }

    public String getAccountHolderName() {
        return this.accountHolderName;
    }
}
```

```

    public String getAccountNumber() {
        return this.accountNumber;
    }

    public double getBalance() {
        return this.balance;
    }

    public LocalDate getOpeningDate() {
        return this.openingDate;
    }

    @Override
    public String toString() {
        return "Inhaber: " + this.getAccountHolderName() +
            "\nKontonummer: " + this.getAccountNumber() +
            "\nEröffnungsdatum: " + this.getOpeningDate() +
            "\nKontostand: " + this.getBalance() + " Euro";
    }

    @Override
    public boolean equals(Object o) {
        if (o == null)
            return false;
        if (!o.getClass().equals(this.getClass()))
            return false;
        BankAccount a = (BankAccount) o;
        return a.getAccountNumber().equals(this.getAccountNumber());
    }
}

```

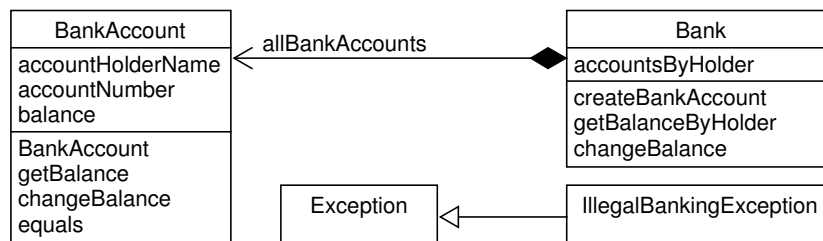
b) (*, 1 Minuten)

Testen Sie Ihre Implementierung mit der im Digicampus zur Verfügung gestellten Klasse `BankAccountMain.java`.

Hinweis: Für diese Teilaufgabe ist keine Abgabe erforderlich. Die von Ihnen erstellte Klasse muss zusammen mit der gegebenen Programmklasse lauffähig sein.

Aufgabe 27 (Eigene Datenklasse implementieren, 25 Minuten)

In dieser Aufgabe sollen Sie eine Klasse implementieren, die das Verhalten einer Bank darstellt.



a) (*, 3 Minuten)

Implementieren Sie eine geprüfte Ausnahme `IllegalBankingException`, die das Setzen einer beliebigen Fehlernachricht erlaubt.

Lösung:

```

public class IllegalBankingException extends Exception {
    public IllegalBankingException(String message) {
        super(message);
    }
}

```

b) (***, 21 Minuten)

Verwenden Sie für die verwalteten Bankkonten die vorgegebene Klasse **BankAccount** (diese wird unten genauer beschrieben). Implementieren Sie eine eigene Datenklasse **Bank** nach folgender Systembeschreibung:

*Eine Bank verwaltet Bankkonten, die einzigartige Kontennummern, einen Inhaber sowie einen Kontostand haben. Eine Bank kann neue Konten eröffnen lassen, dazu bekommt Sie den Namen des Inhabers sowie die gewünschte Kontonummer übergeben. Ist bereits ein Konto mit dieser Kontonummer bei der Bank angelegt worden, so wird eine **IllegalBankingException** geworfen. Die Bank stellt eine Möglichkeit zur Verfügung, die Summe aller Kontostände eines Inhabers zu erhalten. Wird versucht, die Summe aller Kontostände für einen Inhaber zu erhalten, der kein Konto in dieser Bank hat, so wird die Summe 0 zurückgegeben. Außerdem kann von bzw. auf einem Konto mit einer gegebenen Kontonummer Geld abgehoben bzw. eingezahlt werden. Wenn versucht wird, mehr als 500 Euro abzuheben oder einzuzahlen, oder eine nicht existente Kontonummer verwendet wird, so wird eine **IllegalBankingException** geworfen.*

Erzeugen Sie für die Berechnung der Summe aller Kontostände einen geeigneten Stream. Die Klasse **BankAccount** stellt folgende Methoden zur Verfügung:

- **BankAccount(String accountHolderName, String accountNumber)**: Konstruktor, kann **IllegalArgumentException** werfen.
- **boolean equals(Object o)**: Zwei **BankAccount**-Objekte sind genau dann gleich, wenn ihre **accountNumber** gleich ist.
- **String getAccountNumber()**: Gibt die Kontonummer zurück.
- **double getBalance()**: Gibt den Kontostand des Kontos zurück.
- **void changeBalance(double amount)**: Ändert den Kontostand des Kontos um den übergebenen Wert (positiv oder negativ).

Hinweis: Verwenden Sie für die Verwaltung der Bankkonten eine geeignete Collection sowie eine Map, um den Zugriff auf alle Bankkonten eines Inhabers zu erleichtern. Die drei von der Klasse zur Verfügung gestellten Methoden nennen Sie **createBankAccount**, **getBalanceByHolder**, **addBalance**.

Lösung:

```
import java.util.ArrayList;
import java.util.HashMap;
import java.util.HashSet;

public class Bank {
    private final HashMap<String, ArrayList<BankAccount>> accountsByHolder = new HashMap<>();
    private final HashSet<BankAccount> bankAccounts = new HashSet<>();

    public void createBankAccount(String accountHolderName, String accountNumber) throws
    ↪ IllegalBankingException {
        BankAccount temp = new BankAccount(accountHolderName, accountNumber);
        if (!this.bankAccounts.add(temp)) {
            throw new IllegalBankingException("Account number already taken");
        }
        if (!this.accountsByHolder.containsKey(accountHolderName)) {
            this.accountsByHolder.put(accountHolderName, new ArrayList<>());
        }
        this.accountsByHolder.get(accountHolderName).add(temp);
    }

    public double getBalanceByHolder(String accountHolderName) {
        if (!this.accountsByHolder.containsKey(accountHolderName)) {
            return 0;
        }
        return this.accountsByHolder.get(accountHolderName).stream()
            .mapToDouble(BankAccount::getBalance).sum();
    }
}
```

```

    public double addBalance(String accountNumber, double amount) throws
    ↪ IllegalBankingException {
        if (amount > 500 || amount < -500) {
            throw new IllegalBankingException("Can only deposit or withdraw between 0 and 500
            ↪ euros");
        }
        for (BankAccount ba : this.bankAccounts) {
            if (ba.getAccountNumber().equals(accountNumber)) {
                ba.addBalance(amount);
                return amount;
            }
        }
        throw new IllegalBankingException("Account number not taken");
    }
}

```

c) (*, 1 Minuten)

Testen Sie Ihre Implementierung mit der im Digicampus zur Verfügung gestellten Klasse `BankMain.java`.

Hinweis: Für diese Teilaufgabe ist keine Abgabe erforderlich. Die von Ihnen erstellte Klasse muss zusammen mit der gegebenen Programmklasse lauffähig sein.

Aufgabe 28 * (Eigene Datenklassen implementieren, 30 Minuten)

In dieser Aufgabe sollen Sie nach einer Systembeschreibung eine eigene Datenklasse implementieren und diese mit einer eigenen Programmklasse testen.

a) (*, Klasse **Task** nach Systembeschreibung implementieren, 26 Minuten)

Implementieren Sie gemäß folgender Systembeschreibung eine eigene Datenklasse **Task** zur Verwaltung von **Aufgaben**. Versehen Sie dabei die Klasse und alle öffentlichen Konstruktoren und Methoden mit passenden Javadoc-Kommentaren.

Eine Aufgabe hat eine Beschreibung und eine Priorität. Die Beschreibung soll eine nicht-leere Zeichenkette, die nur einmal gesetzt werden kann, sein. Die Priorität soll eine ganze Zahl zwischen 1 und 5 (jeweils einschließlich) sein und jederzeit geändert werden können. Während ungültige Beschreibungen eine geeignete ungeprüfte API-Ausnahme auslösen sollen, soll bei ungültigen Prioritäten automatisch der nächstgelegene gültige Wert gesetzt werden. Es soll die Möglichkeit geben, eine Aufgabe mit spezifischer Beschreibung und Priorität anzulegen, aber auch die Möglichkeit, eine Aufgabe mit spezifischer Beschreibung und vorgegebener Priorität 1 anzulegen. Zwei Aufgaben sollen gleich sein, wenn sie die gleiche Beschreibung ohne Beachtung von Groß- und Kleinschreibung haben. Die Ausgabe einer Aufgabe als Zeichenkette soll deren Beschreibung und Priorität beinhalten in der Form <Beschreibung>(<Priorität>).

Lösung:

```

/**
 * A class representing tasks which consist of a description and a priority.
 */
public class Task {

    private String description;
    private int priority;

```

```

/**
 * Creates a new Task with the specified description and priority.
 *
 * @param description the description of the Task. It must not be null or empty.
 * @param priority    the priority of the Task. Range: [1,5]. Smaller or bigger
 *                   values will be set to 1 or 5 respectively.
 *
 * @throws IllegalArgumentException if the description is either null or empty).
 */
public Task(String description, int priority) {
    setDescription(description);
    setPriority(priority);
}

/**
 * Creates a new Task with the specified description and priority 1.
 *
 * @param description the description of the Task. It must not be null or empty.
 *
 * @throws IllegalArgumentException if the description is either null or empty).
 */
public Task(String description) {
    this(description, 1);
}

/**
 * Returns this Task's description.
 *
 * @return this Task's description.
 */
public String getDescription() {
    return description;
}

private void setDescription(String description) {
    if (!checkDescription(description))
        throw new IllegalArgumentException("Description must not be null or empty");
    else
        this.description = description;
}

private static boolean checkDescription(String description) {
    return (description != null && description.length() > 0);
}

/**
 * Returns this Task's priority.
 *
 * @return this Task's priority.
 */
public int getPriority() {
    return priority;
}

/**
 * Sets the priority of this Task. Range: [1,5]. If the specified value is not
 * valid, the nearest valid value is set.
 *
 * @param priority the priority to be set.
 */
public void setPriority(int priority) {
    if (priority >= 1 && priority <= 5)
        this.priority = priority;
    else if (priority < 1)
        this.priority = 1;
    else
        this.priority = 5;
}

```

```

/**
 * Returns a String object representing this Task's description and priority.
 *
 * @return String representation of this Task's description and priority.
 */
@Override
public String toString() {
    return getDescription() + "(" + getPriority() + ")";
}

/**
 * Compares this Task to the specified object. The result is true if and only if
 * the argument is not null and it is a Task object with the same description
 * (case-insensitive).
 *
 * @return true if the objects are equal; false otherwise.
 */
@Override
public boolean equals(Object o) {
    if (o != null && this.getClass().equals(o.getClass())) {
        Task t = (Task) o;
        if (t.getDescription().equalsIgnoreCase(this.getDescription()))
            return true;
    }
    return false;
}
}

```

b) (*, Programmklasse *TaskApp* zum Test von *Task* implementieren, 4 Minuten)

Implementieren Sie gemäß folgender Vorgaben eine eigene Programmklasse **TaskApp** zum Test von **Task**:

- Erstellen Sie eine Aufgabe mit der Beschreibung **Laufen** und vorgegebener Priorität, ändern Sie dann die Priorität auf 2 und geben deren Repräsentation als String aus.
- Erstellen Sie eine Aufgabe mit der Beschreibung **Uebungsblatt** und Priorität 5 und geben Sie dann deren Repräsentation als String aus.
- Erstellen Sie eine Aufgabe mit der Beschreibung **Einkaufen** und Priorität 0 und geben Sie dann deren Repräsentation als String aus.
- Erstellen Sie eine Aufgabe mit der Beschreibung **einkaufen** und Priorität 3, vergleichen Sie dieses Objekt mit dem zuvor erstellten Objekt und geben Sie dann das Ergebnis dieses Vergleichs aus.

Lösung:

```

public class TaskApp {
    public static void main(String[] args) {
        Task task1 = new Task("Laufen");
        task1.setPriority(2);
        System.out.println(task1);

        Task task2 = new Task("Uebungsblatt", 5);
        System.out.println(task2);

        Task task3 = new Task("Einkaufen", 0);
        System.out.println(task3);

        Task task4 = new Task("einkaufen", 3);
        System.out.println(task4);
        System.out.println(task3.equals(task4));
    }
}

```